ada

BE-JV-005

Aula 03 - Programação Funcional

Revisão Aula 02
Kahoot

Programação/Paradigma Funcional

- Disponível no Java desde a versão 8
- Lambdas e Functional Interfaces
- Podemos agora definir/parametrizar comportamentos

Interfaces funcionais padrão

Interface Padrão	Uso	Entrada	Saída	Método da interface
Consumer<T>	Processar um objeto sem produzir uma resposta	Objeto de tipo T	não possui (Void)	.accept(...)
Supplier<T>	Produzir algum objeto	não possui	Objeto de tipo T	.get()
Function<T, R>	Transformar um objeto em outro	Objeto de tipo T	Objeto de tipo R	.apply(...)
Predicate<T>	Validar a entrada ou definir regras para um objeto	Objeto de tipo T	*boolean*	.test(...)

Fixação

Exercícios

IniciadorDeAtendimento.java

→ Vamos iniciar criando um método para iniciar atendimento que recebe como parâmetro um produtor de protocolo ele não receberá nenhum parâmetro retornará a String de um novo protocolo. Esse primeiro gerador de protocolo produzirá protocolos seguindo a seguinte regra:

- Inicia com PROT-
- Seguido da data e hora atual sem traços
  Exemplo: Atendimento iniciado dia 10/03/2026 às 20:00:10 seria 20260310200010-
- Seguido de um número inteiro aleatório de 0 a 9999.

Exemplo:

Iniciando atendimento com o protocolo PROT-20260310200010-775

ada

Exercícios

IniciadorDeAtendimento.java

→ Vamos agora para exercício anterior implementar um novo supplier de protocolos que agora retorna a string de um identificador UUID aleatório.

Exemplo:

```
Iniciando atendimento com o protocolo 1a3384b6-05bd-46eb-9c45-6b48c2fcc0cd
```

ada

Exercícios

NotificadorDeSms.java

→ Quando um cliente usa o cartão de crédito, o banco dispara um aviso para o celular dele. Crie um comportamento de "Consumo": ele deve receber um cliente como parâmetro de entrada, utilizá-lo para montar uma mensagem e não devolver nada para o sistema, apenas imprimir no console.

Exemplo:

Cliente[nome: Vini]

"Enviando SMS: Olá Vini, compra aprovada no seu cartão!"

ada

ValidadorDeFraude.java

→ O sistema de prevenção a fraudes precisa aprovar ou bloquear transferências financeiras de forma automática. Ele poderá receber diversas regras diferentes. Crie um comportamento de "Teste": ele deve receber o valor de uma transferência e testar se esse valor é considerado de alto risco. A regra de negócio exige que valores maiores que R$ 10.000,00 sejam barrados.

Exemplo:

```
Transferencia[valor: 1500.00]
Resposta: false (Não é fraude)

Transferencia[valor: 20000.00]
Resposta: true (Suspeita de fraude)
```

ada

Exercícios

GeradorDeEmail.java

→ Quando um novo funcionário entra na corretora, o sistema de RH cria um e-mail corporativo automaticamente baseado no nome e sobrenome dele. Crie um comportamento de "Transformação": ele deve receber o primeiro nome do funcionário e devolvê-lo transformado (tudo em letras minúsculas e com o sufixo corporativo adicionado ao final).

Exemplo:

Colaborador[nome: Vinicius, sobrenome: Cornieri]

vinicius.cornieri@empresa.com.br

ada

Exercícios

SelecionadorClientesParaMarketing.java

→ O time de marketing nos passou uma lista de clientes que devemos selecionar removendo da lista todos os que fizeram uma compra recente nos últimos 90 dias, para iniciarmos uma campanha para os que fazem tempo que não compram. Dica: utilize o método `removeIf` da lista.

Exemplo:

```
Clientes:
    Cliente[nome: Teste1, dataUltimaCompra: 2025-03-05]
    Cliente[nome: Teste2, dataUltimaCompra: 2026-03-08]
    Cliente[nome: Teste3, dataUltimaCompra: 2025-11-15]
    Cliente[nome: Teste4, dataUltimaCompra: 2026-02-22]
    Cliente[nome: Teste5, dataUltimaCompra: 2025-10-19]

Clientes selecionados:
Teste1
Teste3
Teste5
```

ada

Exercícios

SelecionadorClientesParaMarketing.java

→ Do exercício anterior solicitaram agora que informemos os clientes ordenadamente do que comprou a mais tempo até o que comprou mais recentemente.

Exemplo:

```
Clientes:
    Cliente[nome: Teste1, dataUltimaCompra: 2025-03-05]
    Cliente[nome: Teste2, dataUltimaCompra: 2026-03-08]
    Cliente[nome: Teste3, dataUltimaCompra: 2025-11-15]
    Cliente[nome: Teste4, dataUltimaCompra: 2026-02-22]
    Cliente[nome: Teste5, dataUltimaCompra: 2025-10-19]

Clientes selecionados ordenados:
Teste1,2025-03-05
Teste5,2025-10-19
Teste3,2025-11-15
```

ada

Obrig.ada